

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- DEBUGGING & OPTIMISATION TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL2- DEBUGGING & OPTIMISATION TASK
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)</i>		
Student Name:	Deepa preya H	Reg.No.:	192421047
Department:	Information Technology	Date:	01.08.2026

ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.
2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.
3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.
4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.
5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

Code of Conduct:

I DEEPA PREYA H / 192421047 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Identification of Errors / Bugs (20%)	Debugging Approach & Analysis (20%)	Correctness of Debugged Solution (25%)	Optimization Techniques Applied (20%)	Testing and Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided

Q1. Debugging Signed Integer Addition Using 2's Complement

Question:

A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.

1. Identification of Errors / Bugs (20%)

Problem:

The program adds +120 and +50 using **8-bit 2's complement**. The maximum positive value that can be represented is +127. Since:

$$120 + 50 = 170$$

170 is outside the 8-bit signed range (-128 to $+127$), causing **signed integer overflow**.

Binary Addition:

$$+120 = 01111000$$

$$+50 = 00110010$$

$$\begin{array}{r} 01111000 \\ + 00110010 \\ \hline 10101010 \end{array}$$

The result **10101010** represents -86 in 2's complement, which is incorrect for the expected sum of **170**.

Error Identified: Signed overflow has occurred.

2. Debugging Approach & Analysis (20%)

Root Cause Analysis:

- Both operands are **positive**.
- The obtained result is **negative**.
- In 2's complement arithmetic, overflow occurs when:
 - Two positive numbers produce a negative result.
 - Two negative numbers produce a positive result.

Here:

$$\text{Sign}(120) = 0$$

$$\text{Sign}(50) = 0$$

$$\text{Sign}(\text{Result}) = 1$$

Since the operand signs are the same but the result sign is different, overflow is detected.

3. Correctness of Debugged Solution (25%)

Overflow Detection Algorithm:

If ($A_sign == B_sign$) AND ($\text{Result_sign} != A_sign$)

 Overflow = True

Else

 Overflow = False

For the given example:

$$A = +120$$

$$B = +50$$

$$\text{Result} = -86$$

$$\text{Overflow} = \text{TRUE}$$

Instead of accepting -86 , the program should report:

Error: Signed Integer Overflow

Expected mathematical result = 170

Result cannot be represented in 8-bit signed format.

4. Optimization Techniques Applied (20%)

- Implement hardware/software overflow flag checking after every signed addition.
- Use a larger data type (16-bit or 32-bit integer) for large values.
- Validate input values before performing arithmetic operations.
- Prevent incorrect computations by stopping execution when overflow is detected.

5. Testing and Documentation (15%)

Test Case	Expected Result	Overflow
20 + 30	50	No
60 + 40	100	No
120 + 50	170 (Not Representable)	Yes
-100 + -40	-140 (Not Representable)	Yes

Conclusion:

The error is caused by **signed integer overflow** because the sum exceeds the 8-bit 2's complement range. The issue is resolved by detecting overflow using sign-bit comparison and handling it appropriately through an overflow flag or by using a larger integer size. This aligns with the debugging and optimization requirements of the assessment.

Q2. Debugging Ripple Carry Adder Delay

Question:

A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.

1. Identification of Errors / Bugs (20%)

Problem:

The processor experiences high delay because it uses a Ripple Carry Adder (RCA).

Error Identified:

- In an RCA, each full adder must wait for the carry output from the previous stage.
- The carry propagates sequentially from the Least Significant Bit (LSB) to the Most Significant Bit (MSB).
- As the number of bits increases, the total addition time also increases, making it unsuitable for real-time systems.

2. Debugging Approach & Analysis (20%)

Root Cause Analysis:

Example (4-bit Ripple Carry Adder):

Bit0 → Carry0

↓

Bit1 → Carry1

↓

Bit2 → Carry2

↓

Bit3 → Final Carry

- Each stage waits for the previous carry.
- Delay accumulates across all bits.
- The carry propagation chain is the **performance bottleneck**.

3. Correctness of Debugged Solution (25%)

Optimized Solution: Use Carry Look-Ahead Adder (CLA)

A Carry Look-Ahead Adder computes carry signals in parallel using Generate (G) and Propagate (P) signals.

Equations:

Generate (G) = $A \times B$

Propagate (P) = $A \oplus B$

Carry:

$C1 = G0 + P0C0$

$C2 = G1 + P1G0 + P1P0C0$

$C3 = G2 + P2G1 + P2P1G0 + P2P1P0C0$

Corrected Result:

- Carries are generated simultaneously instead of sequentially.
- Addition is completed much faster.

4. Optimization Techniques Applied (20%)

- Replace the Ripple Carry Adder with a Carry Look-Ahead Adder (CLA).
- Compute carry bits in parallel using Generate and Propagate logic.
- Reduce carry propagation delay significantly.
- Improve processor speed and overall performance in real-time applications.

Benefits:

- Faster arithmetic operations.
- Lower execution time.
- Better performance for high-speed processors.

5. Testing and Documentation (15%)

Test Case	Ripple Carry Adder	Carry Look-Ahead Adder
4-bit Addition	Correct but slower	Correct and faster
8-bit Addition	High carry delay	Low delay
16-bit Addition	Delay increases	Fast parallel carry generation

Expected Output:

- Correct arithmetic result in both adders.
- CLA completes the addition faster due to parallel carry computation.

Conclusion

The performance bottleneck is caused by the sequential carry propagation in the Ripple Carry Adder. Replacing it with a Carry Look-Ahead Adder eliminates most of the carry delay by generating carry signals in parallel, resulting in faster execution, improved efficiency, and better suitability for real-time systems. This satisfies the debugging, correctness, optimization, and testing requirements of the assessment rubric.

Q3. Debugging Booth's Multiplication Algorithm

Question:

A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.

1. Identification of Errors / Bugs (20%)

Problem:

The multiplication unit produces incorrect results when multiplying **negative numbers** using Booth's algorithm.

Errors Identified:

- Incorrect **bit-pair checking** (Q_0 and Q_{-1}).
- Improper **arithmetic right shift**.
- Incorrect **sign extension** during shifting.
- Wrong addition/subtraction of the multiplicand.

These errors lead to incorrect signed multiplication results.

2. Debugging Approach & Analysis (20%)

Root Cause Analysis:

Booth's algorithm checks the bit pair (Q_0, Q_{-1}) in every cycle.

$Q_0 Q_{-1}$ Operation

- 00 No operation
- 01 Add Multiplicand (M)
- 10 Subtract Multiplicand (M)
- 11 No operation

After each operation:

- Perform an **arithmetic right shift** on **A, Q, and Q_{-1}** .
- The **sign bit must be preserved** during the shift.

Possible Causes of Error:

- Wrong bit-pair interpretation.
- Logical shift used instead of arithmetic shift.
- Sign bit not extended correctly.

3. Correctness of Debugged Solution (25%)

Correct Booth's Algorithm Steps:

1. Initialize **A = 0, Q = Multiplier, M = Multiplicand, $Q_{-1} = 0$** .
2. Check **Q_0 and Q_{-1}** .
3. Perform the required operation:
 - **01** $\rightarrow A = A + M$

- **10** → $A = A - M$
 - **00 or 11** → **No operation**
4. Perform an **arithmetic right shift** while preserving the sign bit.
 5. Repeat for **n** bits.
 6. The final result is stored in **AQ**.

Corrected Result:

- Correct multiplication for both **positive and negative operands**.
- Proper handling of signed numbers.

4. Optimization Techniques Applied (20%)

- Verify **Q_0 – Q_{-1}** bit-pair logic before every operation.
- Always use an **arithmetic right shift** instead of a logical shift.
- Preserve the **sign bit** through proper sign extension.
- Validate the result using known positive and negative test cases.
- Ensure correct addition/subtraction of the multiplicand.

5. Testing and Documentation (15%)

Test Case	Expected Result	Status
5×3	15	Pass
-5×3	-15	Pass
5×-3	-15	Pass
-5×-3	15	Pass

Expected Output:

- All positive and negative multiplication results are correct.
- No incorrect values due to shift or sign-extension errors.

Conclusion

The incorrect results occur because of **wrong bit-pair checking, improper arithmetic right shifting, or incorrect sign extension** in Booth's algorithm. By correctly interpreting **Q_0 – Q_{-1}** , preserving the sign bit during arithmetic shifts, and performing the appropriate add/subtract operation, the multiplication unit produces accurate signed multiplication results while maintaining efficient performance.

Q4. Debugging Restoring Division Algorithm

Question:

An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.

1. Identification of Errors / Bugs (20%)

Problem:

The embedded system experiences **high execution time** because it uses the **restoring division algorithm**.

Errors Identified:

- After each subtraction, if the partial remainder becomes **negative**, the divisor is **added back (restored)**.
- These repeated restoration operations increase the number of arithmetic operations.
- As a result, the division process becomes slower and less efficient.

2. Debugging Approach & Analysis (20%)

Root Cause Analysis:

Restoring Division Workflow:

1. Shift the dividend and remainder.
2. Subtract the divisor from the remainder.
3. If the remainder is **negative**:
 - Restore the previous value by adding the divisor.
 - Set quotient bit = **0**.
4. If the remainder is **positive**:
 - Keep the result.
 - Set quotient bit = **1**.
5. Repeat for all bits.

Performance Bottleneck:

- Frequent **restoration (addition)** after unsuccessful subtraction increases execution time.

3. Correctness of Debugged Solution (25%)

Optimized Solution: Use Non-Restoring Division

The **Non-Restoring Division** algorithm avoids immediate restoration.

Algorithm Steps:

1. Shift the remainder and quotient.
2. If the previous remainder is **positive**, subtract the divisor.
3. If the previous remainder is **negative**, add the divisor.
4. Set the quotient bit based on the sign of the remainder.
5. Repeat until all bits are processed.
6. Perform a **final correction** only if the remainder is negative.

Corrected Result:

- Produces the same quotient and remainder as the restoring method.
- Eliminates unnecessary restoration after every negative remainder.

4. Optimization Techniques Applied (20%)

- Replace the **Restoring Division** algorithm with the **Non-Restoring Division** algorithm.
- Eliminate repeated restoration operations.
- Reduce the total number of arithmetic operations.
- Improve execution speed and processor efficiency.
- Lower power consumption in embedded systems.

Benefits:

- Faster division.
- Reduced execution time.
- Better performance for real-time embedded applications.

5. Testing and Documentation (15%)

Test Case	Restoring Division	Non-Restoring Division
$20 \div 4$	Correct, slower	Correct, faster
$25 \div 5$	Correct, slower	Correct, faster
$18 \div 3$	Correct, slower	Correct, faster
$15 \div 2$	Quotient = 7, Remainder = 1	Quotient = 7, Remainder = 1

Expected Output:

- Both algorithms produce the **same quotient and remainder**.
- **Non-Restoring Division** completes the operation with fewer arithmetic steps and lower execution time.

Conclusion

The inefficiency is caused by the **repeated restoration steps** in the restoring division algorithm. Replacing it with the **Non-Restoring Division** algorithm eliminates unnecessary restoration operations, reducing execution time while maintaining correct quotient and remainder values. This optimization improves processor performance and is well suited for real-time embedded systems, satisfying all the debugging, optimization, correctness, and testing requirements of the assessment rubric.

Q5. Debugging IEEE 754 Single Precision Floating-Point Addition

Question:

A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

1. Identification of Errors / Bugs (20%)

Problem:

The application produces **inaccurate results** when adding a **very small number** to a **very large number** using **IEEE 754 single-precision floating-point** representation.

Errors Identified:

- Loss of precision during **exponent alignment**.
- **Normalization** may discard significant bits.
- **Rounding errors** occur because single precision has only **23-bit mantissa**.

These factors cause the small number to have little or no effect on the final result.

2. Debugging Approach & Analysis (20%)

Root Cause Analysis:

Example:

Large Number = 1.0×10^{10}

Small Number = 1.0×10^{-5}

Before addition:

- Exponents must be aligned.
- The mantissa of the smaller number is shifted many places to the right.
- Most of its significant bits are lost.
- After normalization and rounding, the small value may disappear.

Result:

The computed sum is almost the same as the large number, causing a loss of accuracy.

3. Correctness of Debugged Solution (25%)

Correct Solution:

1. Align the exponents correctly before addition.
2. Normalize the result after addition.
3. Apply IEEE 754 rounding rules correctly.
4. Use **double precision (64-bit)** when higher accuracy is required.
5. For repeated additions, use improved numerical methods such as **Kahan Summation** to reduce rounding errors.

Corrected Result:

- Higher numerical accuracy.
- Reduced loss of significant digits.
- Correct floating-point computation.

4. Optimization Techniques Applied (20%)

- Replace **single precision (32-bit)** with **double precision (64-bit)** for scientific calculations.
- Use **Kahan Summation Algorithm** or similar techniques to minimize accumulated rounding errors.
- Avoid adding numbers with extremely different magnitudes whenever possible.
- Reorder calculations to reduce precision loss.

Benefits:

- Improved accuracy.
- Reduced rounding errors.
- Better numerical stability for scientific applications.

5. Testing and Documentation (15%)

Test Case	Single Precision	Double Precision
$1.5 + 2.5$	Accurate	Accurate
$1 \times 10^{10} + 1 \times 10^{-5}$	Precision loss	More accurate
Large scientific calculations	Small rounding errors	Reduced rounding errors
Repeated additions	Error accumulates	More stable results

Expected Output:

- Single precision may lose the contribution of very small values.
- Double precision preserves more significant digits and produces more accurate results.

Conclusion

The inaccuracy is caused by exponent alignment, normalization, and rounding errors in IEEE 754 single-precision arithmetic. Using double precision (64-bit) and algorithmic improvements such as Kahan Summation significantly improves numerical accuracy and reduces precision loss. This solution satisfies the debugging, correctness, optimization, and testing requirements of the assessment rubric.